Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 9

по курсу «Методы оптимизации»

«Метод Ньютона и его модификация»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Необходимо реализовать метод Ньютона, а также модифицированный метод с постоянным Гессианом.

2 Результаты

Исходный код программы представлен в листинге 1.

```
using Plots
using LinearAlgebra
using ForwardDiff

rosenbrock(x) = (1.0 - x[1])^2 + 100.0 * (x[2] - x[1]^2)^2
rastrigin(x) = 20.0 + (x[1]^2 - 10.0 * cos(2   * x[1])) + (x[2]^2 -
    10.0 * cos(2   * x[2]))
schwefel(x) = 418.9829 * 2.0 - x[1] * sin(sqrt(abs(x[1]))) - x[2] *
    sin(sqrt(abs(x[2])))

function find_interval(   , h=0.1, factor=2.0, max_iter=20)
    a, fa = 0.0,    (0.0)
    b, fb = h,    (h)
    if fb > fa
        h = -h
        b, fb = h,    (h)
        fb > fa && return (-abs(h), abs(h))
    end
    c = b + h
    for _ in 1:max_iter
        fc =    (c)
        fc > fb && return (min(a, c), max(a, c))
        a, fa, b, fb = b, fb, c, fc
        h *= factor
        c = b + h
    end
    return (a, c)
end

function line_search_1d(f, x, d; tol=1e-8)
      (   ) = f(x .+     .* d)
    a, b = find_interval(   )
    abs(b - a) < 1e-12 && return 0.5 * (a + b)
      _gold   = (sqrt(5.0) - 1.0) / 2.0
```

```
    c = b -  _gold  * (b - a)
    d_pt = a +  _gold  * (b - a)
    fc, fd =  (c),  (d_pt)
    while abs(b - a) > tol
        if fc < fd
            b, d_pt, fd = d_pt, c, fc
            c = b -  _gold  * (b - a)
            fc =  (c)
        else
            a, c, fc = c, d_pt, fd
            d_pt = a +  _gold  * (b - a)
            fd =  (d_pt)
        end
    end
    return 0.5 * (a + b)
end

function newton_minimize(f, x0; max_iter=100, tol=1e-8)
    x = copy(x0)
    trajectory = [copy(x)]
    for _ in 1:max_iter
        g = ForwardDiff.gradient(f, x)
        H = ForwardDiff.hessian(f, x)
        if norm(g) <= tol
            break
        end
        d = try
            -H \ g
        catch
            -g
        end
        x_new = x .+ d
        if f(x_new) >= f(x)
                = line_search_1d(f, x, d)
            x_new = x .+  .* d
        end
        x = x_new
        push!(trajectory, copy(x))
    end
    return x, trajectory
end

function newton_mod_minimize(f, x0; max_iter=100, tol=1e-8)
    x = copy(x0)
    trajectory = [copy(x)]
```

```
    H = ForwardDiff.hessian(f, x)
    H_inv = try
        inv(H)
    catch
        n = length(x)
        Matrix{Float64}(I, n, n)
    end
    for _ in 1:max_iter
        g = ForwardDiff.gradient(f, x)
        if norm(g) <= tol
            break
        end
        d = -H_inv * g
        dot(d, g) >= 0 && (d = -g)
           = line_search_1d(f, x, d)
        x = x .+     .* d
        push!(trajectory, copy(x))
    end
    return x, trajectory
end

function plot_trajectory(f, trajectory, xlim, ylim, title, filename)
    xs = range(xlim[1], xlim[2], length=200)
    ys = range(ylim[1], ylim[2], length=200)
    Z = [f([x, y]) for y in ys, x in xs]
    plt = contour(xs, ys, Z, levels=20)
    tr = trajectory
    plot!(plt, [p[1] for p in tr], [p[2] for p in tr], color=:red, linewidth=2, label="")
    scatter!(plt, [tr[1][1]], [tr[1][2]], color=:green, markersize=10, label="           ")
    scatter!(plt, [tr[end][1]], [tr[end][2]], color=:blue, markersize=10, label="           ")
    title!(plt, title)
    xlabel!(plt, "x_1")
    ylabel!(plt, "x_2")
    savefig(plt, filename)
end

x0_rastrigin = [0.3, 0.3]
x0_schwefel = [400.0, 400.0]
x0_rosenbrock = [-1.5, 2.0]

lab9_dir = @__DIR__
```

```

println("                                    ")
x_opt, traj = newton_minimize(rastrigin, x0_rastrigin)
println("  x* = ", x_opt, "\n  f(x*) = ", rastrigin(x_opt), "\n                    : ", length(traj) - 1)
rx = (min(0.75, minimum(p[1] for p in traj)), max(1.02, maximum(p[1] for p in traj)))
ry = (min(-0.05, minimum(p[2] for p in traj)), max(0.28, maximum(p[2] for p in traj)))
plot_trajectory(rastrigin, traj, rx, ry, "                ", joinpath(lab9_dir, "rastrigin_newton.png"))

println("                                                          ")
x_opt, traj = newton_mod_minimize(rastrigin, x0_rastrigin)
println("  x* = ", x_opt, "\n  f(x*) = ", rastrigin(x_opt), "\n                    : ", length(traj) - 1)
rx = (min(0.75, minimum(p[1] for p in traj)), max(1.02, maximum(p[1] for p in traj)))
ry = (min(-0.05, minimum(p[2] for p in traj)), max(0.28, maximum(p[2] for p in traj)))
plot_trajectory(rastrigin, traj, rx, ry, "                                        ", joinpath(lab9_dir, "rastrigin_newton_mod.png"))

println("                              ")
x_opt, traj = newton_minimize(schwefel, x0_schwefel)
println("  x* = ", x_opt, "\n  f(x*) = ", schwefel(x_opt), "\n                    : ", length(traj) - 1)
plot_trajectory(schwefel, traj, (350, 450), (350, 450), "                              ", joinpath(lab9_dir, "schwefel_newton.png"))

println("                                                    ")
x_opt, traj = newton_mod_minimize(schwefel, x0_schwefel)
println("  x* = ", x_opt, "\n  f(x*) = ", schwefel(x_opt), "\n                    : ", length(traj) - 1)
plot_trajectory(schwefel, traj, (350, 450), (350, 450), "                                                    ", joinpath(lab9_dir, "schwefel_newton_mod.png"))

println("                                    ")
x_opt, traj = newton_minimize(rosenbrock, x0_rosenbrock)
println("  x* = ", x_opt, "\n  f(x*) = ", rosenbrock(x_opt), "\n                    : ", length(traj) - 1)
```

```
plot_trajectory(rosenbrock, traj, (-2, 2), (-1, 5), "
                                    ", joinpath(lab9_dir, "
    rosenbrock_newton.png"))

println("                                                    ")
x_opt, traj = newton_mod_minimize(rosenbrock, x0_rosenbrock)
println("  x* = ", x_opt, "\n  f(x*) = ", rosenbrock(x_opt), "\n
                    : ", length(traj) - 1)
plot_trajectory(rosenbrock, traj, (-2, 2), (-1, 5), "
                                                        ",
    joinpath(lab9_dir, "rosenbrock_newton_mod.png"))
```

Результат представлен на рисунках 1– 3.

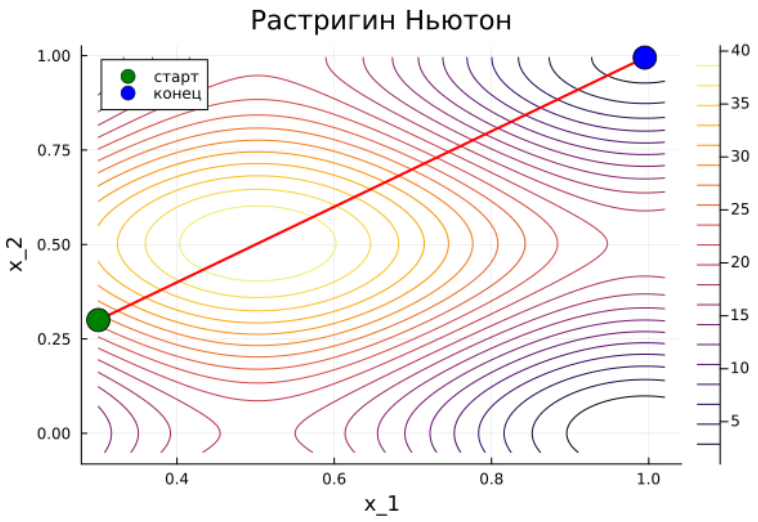

Рис. 1 — Результат

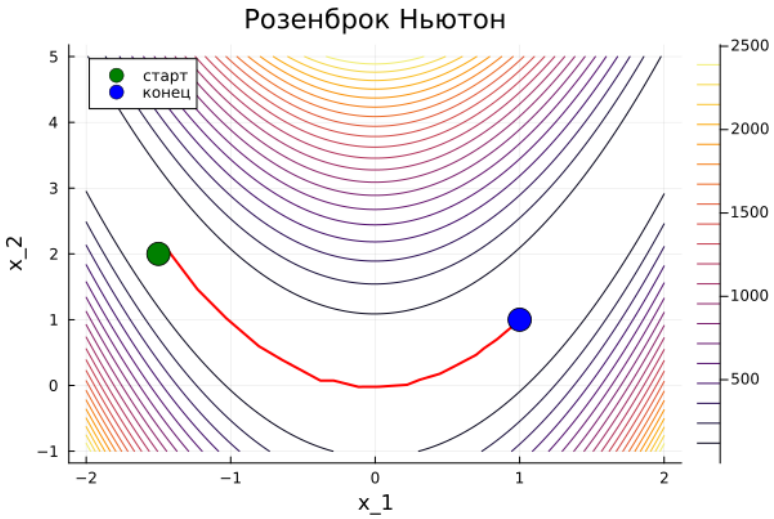

Рис. 2 — Результат

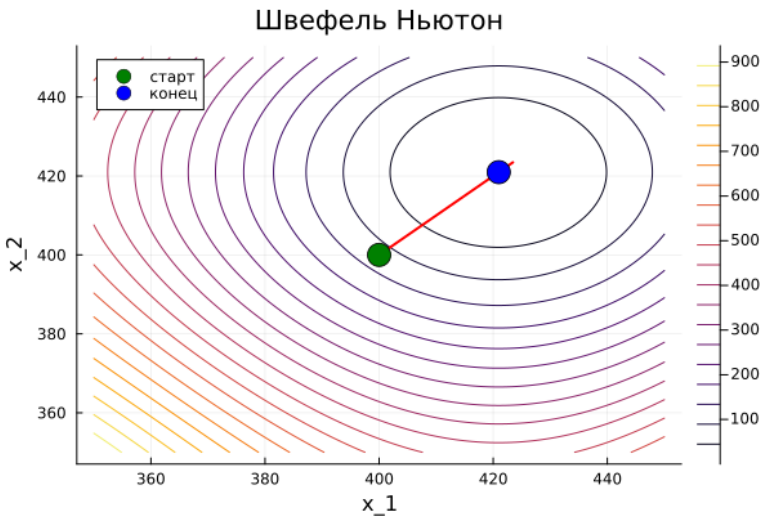

Рис. 3 — Результат